```
-- ======================================================================
-- BUDGETOS — COMPLETE PRODUCTION DATABASE SCHEMA (CORRECTED / IDEMPOTENT)
-- Supabase / PostgreSQL
-- Run this in an empty (or existing) Supabase database. Safe to re-run.
-- ======================================================================

-- ======================================================================
-- 0. CLEAN TEARDOWN (safe re-run — drops in correct dependency order)
-- ======================================================================
DROP VIEW IF EXISTS public.top_spending_categories CASCADE;
DROP VIEW IF EXISTS public.savings_progress CASCADE;
DROP VIEW IF EXISTS public.expense_history CASCADE;
DROP VIEW IF EXISTS public.category_analytics CASCADE;
DROP VIEW IF EXISTS public.monthly_analytics CASCADE;
DROP VIEW IF EXISTS public.dashboard_summary CASCADE;

DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;
DROP TRIGGER IF EXISTS trg_monthly_budgets_allowance_change ON
public.monthly_budgets;
DROP TRIGGER IF EXISTS trg_categories_change ON public.monthly_budget_categories;
DROP TRIGGER IF EXISTS trg_expenses_change ON public.expenses;
DROP TRIGGER IF EXISTS trg_settings_updated_at ON public.settings;
DROP TRIGGER IF EXISTS trg_monthly_budgets_updated_at ON public.monthly_budgets;
DROP TRIGGER IF EXISTS trg_profiles_updated_at ON public.profiles;
DROP TRIGGER IF EXISTS trg_recurring_expenses_updated_at ON
public.recurring_expenses;

DROP FUNCTION IF EXISTS public.handle_new_user() CASCADE;
DROP FUNCTION IF EXISTS public.create_new_month(uuid) CASCADE;
```

```
DROP FUNCTION IF EXISTS public.archive_previous_month(uuid) CASCADE;
DROP FUNCTION IF EXISTS public.create_default_categories(uuid, uuid) CASCADE;
DROP FUNCTION IF EXISTS public.trg_monthly_budgets_allowance_change() CASCADE;
DROP FUNCTION IF EXISTS public.trg_categories_after_change() CASCADE;
DROP FUNCTION IF EXISTS public.trg_expenses_after_change() CASCADE;
DROP FUNCTION IF EXISTS public.recalc_category_and_month(uuid, uuid) CASCADE;
DROP FUNCTION IF EXISTS public.calculate_category_spending(uuid) CASCADE;
DROP FUNCTION IF EXISTS public.calculate_savings_progress(uuid) CASCADE;
DROP FUNCTION IF EXISTS public.calculate_remaining_budget(uuid) CASCADE;
DROP FUNCTION IF EXISTS public.calculate_total_spending(uuid) CASCADE;
DROP FUNCTION IF EXISTS public.set_updated_at() CASCADE;

DROP TABLE IF EXISTS public.expenses CASCADE;
DROP TABLE IF EXISTS public.recurring_expenses CASCADE;
DROP TABLE IF EXISTS public.monthly_budget_categories CASCADE;
DROP TABLE IF EXISTS public.settings CASCADE;
DROP TABLE IF EXISTS public.monthly_budgets CASCADE;
DROP TABLE IF EXISTS public.profiles CASCADE;

-- ========================================================================
-- 1. EXTENSIONS
-- ========================================================================
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- ========================================================================
-- 2. TABLES
-- ========================================================================
```

```
-- profiles
CREATE TABLE public.profiles (
  id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email text NOT NULL,
  full_name text,
  avatar_url text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- monthly_budgets (formerly budget_months) — PK column: id
CREATE TABLE public.monthly_budgets (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  month integer NOT NULL CHECK (month BETWEEN 1 AND 12),
  year integer NOT NULL CHECK (year BETWEEN 2000 AND 2100),
  allowance numeric(12,2) NOT NULL DEFAULT 0 CHECK (allowance >= 0),
  savings_goal numeric(12,2) NOT NULL DEFAULT 0 CHECK (savings_goal >= 0),
  current_balance numeric(12,2) NOT NULL DEFAULT 0,
  remaining_budget numeric(12,2) NOT NULL DEFAULT 0,
  total_spent numeric(12,2) NOT NULL DEFAULT 0 CHECK (total_spent >= 0),
  archived boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT uq_monthly_budgets_user_month_year UNIQUE (user_id, month, year)
);
```

```
-- monthly_budget_categories (formerly categories) — FK: monthly_budget_id -> monthly_budgets.id
CREATE TABLE public.monthly_budget_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  monthly_budget_id uuid NOT NULL REFERENCES public.monthly_budgets(id) ON DELETE CASCADE,
  name text NOT NULL,
  icon text,
  color text,
  budget numeric(12,2) NOT NULL DEFAULT 0 CHECK (budget >= 0),
  spent numeric(12,2) NOT NULL DEFAULT 0 CHECK (spent >= 0),
  remaining numeric(12,2) NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- expenses — FK: monthly_budget_id -> monthly_budgets.id, category_id -> monthly_budget_categories.id
CREATE TABLE public.expenses (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  category_id uuid NOT NULL REFERENCES public.monthly_budget_categories(id) ON DELETE CASCADE,
  monthly_budget_id uuid NOT NULL REFERENCES public.monthly_budgets(id) ON DELETE CASCADE,
  amount numeric(12,2) NOT NULL CHECK (amount > 0),
  description text,
  expense_date date NOT NULL DEFAULT current_date,
  payment_method text,
```

```
  notes text,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- recurring_expenses (new) — templates that generate expenses on a schedule
CREATE TABLE public.recurring_expenses (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  category_id uuid NOT NULL REFERENCES public.monthly_budget_categories(id) ON DELETE CASCADE,
  amount numeric(12,2) NOT NULL CHECK (amount > 0),
  description text,
  payment_method text,
  notes text,
  frequency text NOT NULL DEFAULT 'monthly' CHECK (frequency IN ('daily','weekly','biweekly','monthly','yearly')),
  start_date date NOT NULL DEFAULT current_date,
  next_due_date date NOT NULL DEFAULT current_date,
  end_date date,
  active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- settings
CREATE TABLE public.settings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE,
  currency text NOT NULL DEFAULT 'INR',
```

```
  theme text NOT NULL DEFAULT 'light' CHECK (theme IN ('light','dark','system')),
  notifications boolean NOT NULL DEFAULT true,
  first_day_of_month integer NOT NULL DEFAULT 1 CHECK (first_day_of_month BETWEEN 1 AND 31),
  default_allowance numeric(12,2) NOT NULL DEFAULT 2500 CHECK (default_allowance >= 0),
  default_savings_goal numeric(12,2) NOT NULL DEFAULT 500 CHECK (default_savings_goal >= 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- ======================================================================
-- 3. INDEXES
-- ======================================================================
CREATE INDEX idx_monthly_budgets_user_id ON public.monthly_budgets(user_id);
CREATE INDEX idx_monthly_budgets_month ON public.monthly_budgets(month);
CREATE INDEX idx_monthly_budgets_year ON public.monthly_budgets(year);
CREATE INDEX idx_monthly_budgets_user_year_month ON public.monthly_budgets(user_id, year, month);

CREATE INDEX idx_monthly_budget_categories_user_id ON public.monthly_budget_categories(user_id);
CREATE INDEX idx_monthly_budget_categories_monthly_budget_id ON public.monthly_budget_categories(monthly_budget_id);

CREATE INDEX idx_expenses_user_id ON public.expenses(user_id);
CREATE INDEX idx_expenses_category_id ON public.expenses(category_id);
CREATE INDEX idx_expenses_monthly_budget_id ON public.expenses(monthly_budget_id);
CREATE INDEX idx_expenses_expense_date ON public.expenses(expense_date);
```

```
CREATE INDEX idx_recurring_expenses_user_id ON public.recurring_expenses(user_id);

CREATE INDEX idx_recurring_expenses_category_id ON
public.recurring_expenses(category_id);

CREATE INDEX idx_recurring_expenses_next_due_date ON
public.recurring_expenses(next_due_date);


CREATE INDEX idx_settings_user_id ON public.settings(user_id);


-- ============================================================================

-- 4. GENERIC updated_at TRIGGER FUNCTION

-- ============================================================================

CREATE OR REPLACE FUNCTION public.set_updated_at()

RETURNS trigger

LANGUAGE plpgsql

AS $$

BEGIN

  NEW.updated_at = now();

  RETURN NEW;

END;

$$;


CREATE TRIGGER trg_profiles_updated_at

BEFORE UPDATE ON public.profiles

FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();


CREATE TRIGGER trg_monthly_budgets_updated_at

BEFORE UPDATE ON public.monthly_budgets

FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
```

```
CREATE TRIGGER trg_settings_updated_at
BEFORE UPDATE ON public.settings
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

CREATE TRIGGER trg_recurring_expenses_updated_at
BEFORE UPDATE ON public.recurring_expenses
FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

-- ============================================================
-- 5. CORE CALCULATION FUNCTIONS
-- ============================================================
CREATE OR REPLACE FUNCTION public.calculate_total_spending(p_monthly_budget_id uuid)
RETURNS numeric
LANGUAGE sql
STABLE
AS $$
  SELECT COALESCE(SUM(amount), 0)
  FROM public.expenses
  WHERE monthly_budget_id = p_monthly_budget_id;
$$;

CREATE OR REPLACE FUNCTION public.calculate_remaining_budget(p_monthly_budget_id
uuid)
RETURNS numeric
LANGUAGE sql
STABLE
AS $$
```

```
  SELECT COALESCE(mb.allowance, 0) - COALESCE(public.calculate_total_spending(mb.id), 0)
  FROM public.monthly_budgets mb
  WHERE mb.id = p_monthly_budget_id;
$$;

CREATE OR REPLACE FUNCTION public.calculate_savings_progress(p_monthly_budget_id
uuid)
RETURNS numeric
LANGUAGE sql
STABLE
AS $$
  SELECT CASE
    WHEN mb.savings_goal IS NULL OR mb.savings_goal = 0 THEN 0
    ELSE LEAST(100, GREATEST(0,
      ROUND((public.calculate_remaining_budget(mb.id) / mb.savings_goal) * 100, 2)
    ))
  END
  FROM public.monthly_budgets mb
  WHERE mb.id = p_monthly_budget_id;
$$;

CREATE OR REPLACE FUNCTION public.calculate_category_spending(p_category_id uuid)
RETURNS numeric
LANGUAGE sql
STABLE
AS $$
  SELECT COALESCE(SUM(amount), 0)
  FROM public.expenses
```

```
  WHERE category_id = p_category_id;
$$;

-- ====================================================================
-- 6. RECALCULATION TRIGGERS
-- ====================================================================
CREATE OR REPLACE FUNCTION public.recalc_category_and_month(p_category_id uuid,
p_monthly_budget_id uuid)
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE public.monthly_budget_categories
  SET spent = public.calculate_category_spending(p_category_id),
      remaining = budget - public.calculate_category_spending(p_category_id)
  WHERE id = p_category_id;

  UPDATE public.monthly_budgets
  SET total_spent = public.calculate_total_spending(p_monthly_budget_id),
      remaining_budget = allowance - public.calculate_total_spending(p_monthly_budget_id),
      current_balance = allowance - public.calculate_total_spending(p_monthly_budget_id)
  WHERE id = p_monthly_budget_id;
END;
$$;

CREATE OR REPLACE FUNCTION public.trg_expenses_after_change()
RETURNS trigger
LANGUAGE plpgsql
```

```
AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    PERFORM public.recalc_category_and_month(NEW.category_id,
NEW.monthly_budget_id);
    RETURN NEW;
  ELSIF TG_OP = 'UPDATE' THEN
    PERFORM public.recalc_category_and_month(OLD.category_id, OLD.monthly_budget_id);
    PERFORM public.recalc_category_and_month(NEW.category_id,
NEW.monthly_budget_id);
    RETURN NEW;
  ELSIF TG_OP = 'DELETE' THEN
    PERFORM public.recalc_category_and_month(OLD.category_id, OLD.monthly_budget_id);
    RETURN OLD;
  END IF;
  RETURN NULL;
END;
$$;

CREATE TRIGGER trg_expenses_change
AFTER INSERT OR UPDATE OR DELETE ON public.expenses
FOR EACH ROW EXECUTE FUNCTION public.trg_expenses_after_change();

CREATE OR REPLACE FUNCTION public.trg_categories_after_change()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF TG_OP = 'UPDATE' THEN
```

```
    UPDATE public.monthly_budget_categories
    SET remaining = NEW.budget - NEW.spent
    WHERE id = NEW.id;
  END IF;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_categories_change
BEFORE UPDATE OF budget ON public.monthly_budget_categories
FOR EACH ROW EXECUTE FUNCTION public.trg_categories_after_change();

CREATE OR REPLACE FUNCTION public.trg_monthly_budgets_allowance_change()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  NEW.remaining_budget = NEW.allowance - NEW.total_spent;
  NEW.current_balance = NEW.allowance - NEW.total_spent;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_monthly_budgets_allowance_change
BEFORE UPDATE OF allowance ON public.monthly_budgets
FOR EACH ROW EXECUTE FUNCTION public.trg_monthly_budgets_allowance_change();

-- ================================================================
```

```
-- 7. DEFAULT CATEGORY SEEDING FUNCTION
-- ============================================================

CREATE OR REPLACE FUNCTION public.create_default_categories(p_user_id uuid,
p_monthly_budget_id uuid)
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO public.monthly_budget_categories (user_id, monthly_budget_id, name, icon,
color, budget, spent, remaining)
  VALUES
    (p_user_id, p_monthly_budget_id, 'Food', 'utensils', '#F97316', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'College', 'graduation-cap', '#3B82F6', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Transport', 'car', '#22C55E', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Shopping', 'shopping-bag', '#EC4899', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Subscriptions', 'repeat', '#8B5CF6', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Entertainment', 'film', '#EAB308', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Health', 'heart-pulse', '#EF4444', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Bills', 'file-text', '#06B6D4', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Travel', 'plane', '#14B8A6', 0, 0, 0),
    (p_user_id, p_monthly_budget_id, 'Misc', 'box', '#64748B', 0, 0, 0);
END;
$$;


-- ============================================================
-- 8. ARCHIVE PREVIOUS MONTH FUNCTION
-- ============================================================

CREATE OR REPLACE FUNCTION public.archive_previous_month(p_user_id uuid)
RETURNS void
```

```
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE public.monthly_budgets
  SET archived = true
  WHERE user_id = p_user_id
    AND archived = false
    AND (year < EXTRACT(year FROM current_date)::int
        OR (year = EXTRACT(year FROM current_date)::int
            AND month < EXTRACT(month FROM current_date)::int));
END;
$$;


-- ========================================================================
-- 9. CREATE NEW MONTH FUNCTION
-- ========================================================================
CREATE OR REPLACE FUNCTION public.create_new_month(p_user_id uuid)
RETURNS uuid
LANGUAGE plpgsql
AS $$
DECLARE
  v_month integer := EXTRACT(month FROM current_date)::int;
  v_year integer := EXTRACT(year FROM current_date)::int;
  v_monthly_budget_id uuid;
  v_allowance numeric(12,2);
  v_savings_goal numeric(12,2);
BEGIN
  SELECT id INTO v_monthly_budget_id
```

```
FROM public.monthly_budgets
WHERE user_id = p_user_id AND month = v_month AND year = v_year;

IF v_monthly_budget_id IS NOT NULL THEN
  RETURN v_monthly_budget_id;
END IF;

PERFORM public.archive_previous_month(p_user_id);

SELECT default_allowance, default_savings_goal
INTO v_allowance, v_savings_goal
FROM public.settings
WHERE user_id = p_user_id;

IF v_allowance IS NULL THEN
  v_allowance := 2500;
END IF;
IF v_savings_goal IS NULL THEN
  v_savings_goal := 500;
END IF;

INSERT INTO public.monthly_budgets (
  user_id, month, year, allowance, savings_goal,
  current_balance, remaining_budget, total_spent, archived
)
VALUES (
  p_user_id, v_month, v_year, v_allowance, v_savings_goal,
  v_allowance, v_allowance, 0, false
```

```
  )
  RETURNING id INTO v_monthly_budget_id;

  PERFORM public.create_default_categories(p_user_id, v_monthly_budget_id);

  RETURN v_monthly_budget_id;
END;
$$;

-- ========================================================================
-- 10. AUTOMATIC NEW USER SETUP
-- ========================================================================
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, avatar_url)
  VALUES (
    NEW.id,
    NEW.email,
    COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.raw_user_meta_data-
>>'name'),
    NEW.raw_user_meta_data->>'avatar_url'
  )
  ON CONFLICT (id) DO NOTHING;
```

```
  INSERT INTO public.settings (user_id, currency, theme, notifications, first_day_of_month, default_allowance, default_savings_goal)
  VALUES (NEW.id, 'INR', 'light', true, 1, 2500, 500)
  ON CONFLICT (user_id) DO NOTHING;

  PERFORM public.create_new_month(NEW.id);

  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- ============================================================
-- 11. ROW LEVEL SECURITY
-- ============================================================
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.monthly_budgets ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.monthly_budget_categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.expenses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.recurring_expenses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.settings ENABLE ROW LEVEL SECURITY;

CREATE POLICY "profiles_select_own" ON public.profiles FOR SELECT TO authenticated
USING (auth.uid() = id);
```

```
CREATE POLICY "profiles_insert_own" ON public.profiles FOR INSERT TO authenticated WITH CHECK (auth.uid() = id);

CREATE POLICY "profiles_update_own" ON public.profiles FOR UPDATE TO authenticated USING (auth.uid() = id) WITH CHECK (auth.uid() = id);

CREATE POLICY "profiles_delete_own" ON public.profiles FOR DELETE TO authenticated USING (auth.uid() = id);


CREATE POLICY "monthly_budgets_select_own" ON public.monthly_budgets FOR SELECT TO authenticated USING (auth.uid() = user_id);

CREATE POLICY "monthly_budgets_insert_own" ON public.monthly_budgets FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);

CREATE POLICY "monthly_budgets_update_own" ON public.monthly_budgets FOR UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "monthly_budgets_delete_own" ON public.monthly_budgets FOR DELETE TO authenticated USING (auth.uid() = user_id);


CREATE POLICY "monthly_budget_categories_select_own" ON public.monthly_budget_categories FOR SELECT TO authenticated USING (auth.uid() = user_id);

CREATE POLICY "monthly_budget_categories_insert_own" ON public.monthly_budget_categories FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);

CREATE POLICY "monthly_budget_categories_update_own" ON public.monthly_budget_categories FOR UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "monthly_budget_categories_delete_own" ON public.monthly_budget_categories FOR DELETE TO authenticated USING (auth.uid() = user_id);


CREATE POLICY "expenses_select_own" ON public.expenses FOR SELECT TO authenticated USING (auth.uid() = user_id);

CREATE POLICY "expenses_insert_own" ON public.expenses FOR INSERT TO authenticated WITH CHECK (auth.uid() = user_id);
```

```
CREATE POLICY "expenses_update_own" ON public.expenses FOR UPDATE TO authenticated
USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "expenses_delete_own" ON public.expenses FOR DELETE TO authenticated
USING (auth.uid() = user_id);


CREATE POLICY "recurring_expenses_select_own" ON public.recurring_expenses FOR
SELECT TO authenticated USING (auth.uid() = user_id);

CREATE POLICY "recurring_expenses_insert_own" ON public.recurring_expenses FOR INSERT
TO authenticated WITH CHECK (auth.uid() = user_id);

CREATE POLICY "recurring_expenses_update_own" ON public.recurring_expenses FOR
UPDATE TO authenticated USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "recurring_expenses_delete_own" ON public.recurring_expenses FOR
DELETE TO authenticated USING (auth.uid() = user_id);


CREATE POLICY "settings_select_own" ON public.settings FOR SELECT TO authenticated
USING (auth.uid() = user_id);

CREATE POLICY "settings_insert_own" ON public.settings FOR INSERT TO authenticated
WITH CHECK (auth.uid() = user_id);

CREATE POLICY "settings_update_own" ON public.settings FOR UPDATE TO authenticated
USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "settings_delete_own" ON public.settings FOR DELETE TO authenticated
USING (auth.uid() = user_id);


-- ======================================================================

-- 12. VIEWS

-- ======================================================================

CREATE VIEW public.dashboard_summary

WITH (security_invoker = true) AS

SELECT

  mb.id AS monthly_budget_id,

  mb.user_id, mb.month, mb.year, mb.allowance, mb.savings_goal,
```

```
  mb.current_balance, mb.remaining_budget, mb.total_spent, mb.archived,
  public.calculate_savings_progress(mb.id) AS savings_progress_pct,
  (SELECT COUNT(*) FROM public.monthly_budget_categories c WHERE
c.monthly_budget_id = mb.id) AS category_count,
  (SELECT COUNT(*) FROM public.expenses e WHERE e.monthly_budget_id = mb.id) AS
expense_count
FROM public.monthly_budgets mb;

CREATE VIEW public.monthly_analytics
WITH (security_invoker = true) AS
SELECT
  mb.id AS monthly_budget_id,
  mb.user_id, mb.month, mb.year, mb.allowance, mb.total_spent, mb.remaining_budget,
mb.savings_goal,
  public.calculate_savings_progress(mb.id) AS savings_progress_pct,
  CASE WHEN mb.allowance > 0 THEN ROUND((mb.total_spent / mb.allowance) * 100, 2)
ELSE 0 END AS spent_pct_of_allowance,
  mb.archived
FROM public.monthly_budgets mb
ORDER BY mb.year DESC, mb.month DESC;

CREATE VIEW public.category_analytics
WITH (security_invoker = true) AS
SELECT
  c.id AS category_id, c.user_id, c.monthly_budget_id, c.name, c.icon, c.color,
  c.budget, c.spent, c.remaining,
  CASE WHEN c.budget > 0 THEN ROUND((c.spent / c.budget) * 100, 2) ELSE 0 END AS
spent_pct_of_budget,
  (SELECT COUNT(*) FROM public.expenses e WHERE e.category_id = c.id) AS expense_count
```

```
FROM public.monthly_budget_categories c;


CREATE VIEW public.expense_history
WITH (security_invoker = true) AS
SELECT
  e.id AS expense_id, e.user_id, e.monthly_budget_id, mb.month, mb.year,
  e.category_id, c.name AS category_name, c.icon AS category_icon, c.color AS category_color,
  e.amount, e.description, e.expense_date, e.payment_method, e.notes, e.created_at
FROM public.expenses e
JOIN public.monthly_budget_categories c ON c.id = e.category_id
JOIN public.monthly_budgets mb ON mb.id = e.monthly_budget_id
ORDER BY e.expense_date DESC, e.created_at DESC;


CREATE VIEW public.savings_progress
WITH (security_invoker = true) AS
SELECT
  mb.id AS monthly_budget_id, mb.user_id, mb.month, mb.year, mb.savings_goal, mb.remaining_budget,
  public.calculate_savings_progress(mb.id) AS progress_pct,
  CASE WHEN mb.remaining_budget >= mb.savings_goal THEN true ELSE false END AS goal_achieved
FROM public.monthly_budgets mb;


CREATE VIEW public.top_spending_categories
WITH (security_invoker = true) AS
SELECT
  c.id AS category_id, c.user_id, c.monthly_budget_id, c.name, c.icon, c.color, c.spent, c.budget,
```

```
  RANK() OVER (PARTITION BY c.monthly_budget_id ORDER BY c.spent DESC) AS
spending_rank

FROM public.monthly_budget_categories c

ORDER BY c.monthly_budget_id, spending_rank;


-- ======================================================================

-- END OF SCRIPT

-- ======================================================================
```